



Quik-do

Documentation

By

Ghassan Elgendy

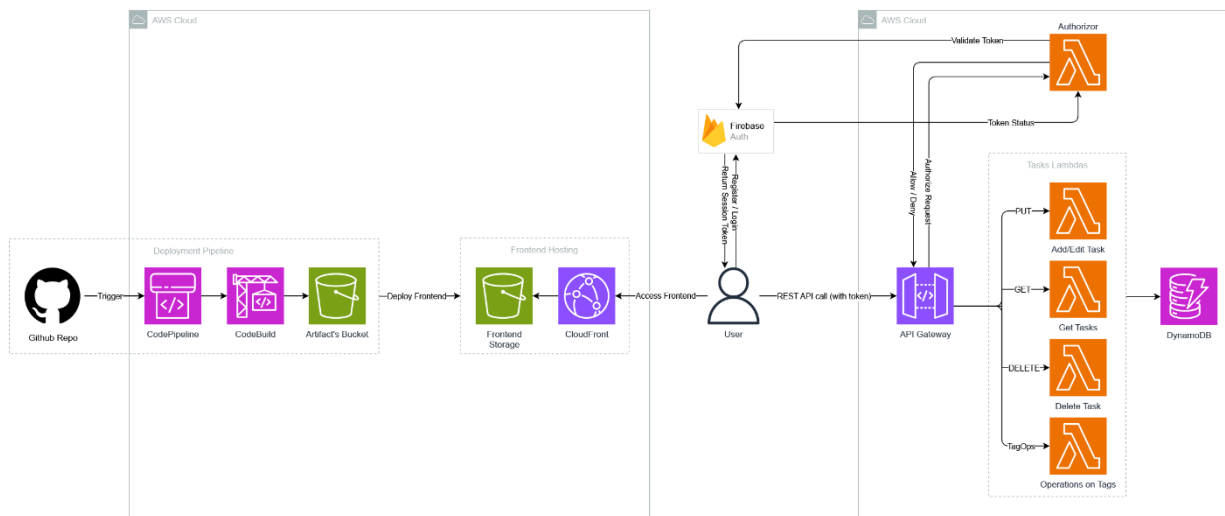
1. Application Overview

Quik-do is a simple and modern to-do list application that allows users to manage their tasks. It provides features like creating, editing, deleting, and marking tasks as complete. The application is designed to be fast, responsive, and accessible from any device with a web browser.

2. Architecture

The application follows a serverless architecture, leveraging various AWS services to provide a scalable and cost-effective solution.

2.1. High-Level Architecture Diagram



2.2. Component Communication

- 1. User Interaction:** The user interacts with the Quik-do application through a web browser.

2. **Content Delivery:** The static assets of the application (HTML, CSS, JavaScript) are hosted on an Amazon S3 bucket and served to the user through Amazon CloudFront, a Content Delivery Network (CDN), for low-latency access.
3. **Authentication:** The application uses Firebase Authentication to manage user sign-up, sign-in, and session management.
4. **API Requests:** Once authenticated, the frontend application makes API requests to an Amazon API Gateway endpoint to perform CRUD (Create, Read, Update, Delete) operations on the to-do items.
5. **Backend Logic:** The API Gateway endpoint triggers AWS Lambda functions that contain the business logic for handling the API requests.
6. **Data Persistence:** The Lambda functions interact with an Amazon DynamoDB database to store and retrieve the to-do list data.

3. Frontend

- **Framework:** The frontend is a single-page application (SPA) built with **React**.
- **Hosting:** The application is hosted on **Amazon S3** as a static website. The S3 bucket is configured for website hosting and is publicly accessible.
- **Deployment:** The application is built and deployed using **AWS CodeBuild**. This ensures a consistent and automated deployment process.

4. Backend

The backend is completely serverless, which means there are no servers to manage.

- **API:** The backend is exposed through a REST API managed by **Amazon API Gateway**. The API Gateway endpoint is `https://by489qc8yj.execute-api.eu-west-1.amazonaws.com`.
- **Authentication: Firebase Authentication** is used to secure the API and manage user identities. The frontend application obtains a JWT token from Firebase after authentication, which is then used to authorize API requests.
- **Business Logic:** The business logic is implemented in **AWS Lambda** functions. These functions are triggered by the API Gateway when a request is received.
- **Database:** The application data is stored in **Amazon DynamoDB**, a NoSQL database that provides fast and predictable performance with seamless scalability.

5. API Endpoints

The following are the main API endpoints used by the application:

5.1. Todos

GET /todos

- **Description:** Retrieves all to-do items for the authenticated user.
- **Request:**
 - **Headers:**
 - Authorization: Bearer <FIREBASE_ID_TOKEN>
- **Response:**
 - **200 OK:**

```
[
  {
    "id": "<TODO_ID>",
    "title": "<TODO_TITLE>",
```

```

    "description": "<TODO_DESCRIPTION>",
    "completed": false,
    "createdAt": "<TIMESTAMP>",
    "updatedAt": "<TIMESTAMP>"
  }
]

```

PUT /todos

- **Description:** Creates or updates a to-do item.
- **Request:**
 - **Headers:**
 - Authorization: Bearer <FIREBASE_ID_TOKEN>
 - **Body:**

```

{
  "id": "<TODO_ID>", // Optional, for updating
existing items
  "title": "<TODO_TITLE>",
  "description": "<TODO_DESCRIPTION>",
  "completed": false
}

```

- **Response:**
 - **200 OK:**

```

{
  "id": "<TODO_ID>",
  "title": "<TODO_TITLE>",
  "description": "<TODO_DESCRIPTION>",
  "completed": false,
  "createdAt": "<TIMESTAMP>",
  "updatedAt": "<TIMESTAMP>"
}

```

DELETE /todos/{id}

- **Description:** Deletes a to-do item.
- **Request:**
 - **Headers:**
 - Authorization: Bearer <FIREBASE_ID_TOKEN>
- **Response:**
 - **204 No Content**

5.2. Tags

GET /tags

- **Description:** Retrieves all tags for the authenticated user.
- **Request:**
 - **Headers:**
 - Authorization: Bearer <FIREBASE_ID_TOKEN>
- **Response:**
 - **200 OK:**

```
[  
  {  
    "id": "<TAG_ID>",  
    "name": "<TAG_NAME>"  
  }  
]
```

DELETE /tags/{id}

- **Description:** Deletes a tag.
- **Request:**
 - **Headers:**
 - Authorization: Bearer <FIREBASE_ID_TOKEN>
- **Response:**
 - **204 No Content**

6. Deployment

The application is deployed using a CI/CD pipeline configured with **AWS CodeBuild**. The pipeline is triggered by a push to the 'frontend' branch of the GitHub repository. The build process installs the dependencies, builds the React application, and deploys the static assets to the S3 bucket.